Autonomous and Adaptive Systems - Project Report

Anonymous Author(s)
Affiliation
Address
email

1 Introduction

For this project, I choose to implement and train an agent to solve the StagHunt Environment using the algorithm REINFORCE because it is the first policy gradient method [cite the REINFORCE paper], and I considered it a good starting point for this project. I choose to implement the REINFORCE with baseline version to improve convergence and reduce noise (even though we still need to generate full episodes because REINFORCE is still a Monte Carlo Method)

2 Implementation

We begin by analyzing the task environment following the classification introduced by Russel and Norvig [cite "AIMA"]: fully observable, multiagent, cooperative, nondeterministic (the deer moves randomly), sequential(this needs explaining the difference between "sequential" in Russel and Norvig vs Stuart and Burton), dynamic, discrete and unknown (the agent recives only a reward, the knowledge of how that reward is calculated is not included in its knowledge base).

Data la dimensione del problema e i dati in input: one hot encoding per 25 celle e 5 entita', iniziamo con una rete "semplice" con 3 layers: 3 linear layers associated with 3 ReLU layers whit dimensions going from 125 -> (125, 64) -> (64,64) -> (64, 5) and then modify it in response from the training performance (since REINFORCE has high variance, we try to keep a low number of parameters)

Baseline rewards for staghunt with fully random moves: [-150.0, -170.0]

epoch	total reward player 1	policy loss player 1	total reward player 2	policy loss player 2
1	-8.0	-3.9738977	-17.0	-9.742985
2	-8.0	-3.9356263	-19.0	-13.2137575
3	-13.0	-2.9319391	-24.0	-14.494476
4	1.0	0.81833726	-24.0	-5.604617
5	-5.0	0.26126307	3.0	1.693016
6	-10.0	-3.5559564	-5.0	-0.14945991
7	-10.0	-2.4129531	-15.0	-1.8485566
8	-5.0	-0.4148963	-20.0	-0.7428379
9	-10.0	0.080888875	-25.0	-6.0461183
10	-50.0	-0.0023919016	-5.0	1.1740055

Table 1: First Training with REINFORCE with baseline

The first agent seems to be exploring very few strategies (hence its low score with relatively stable policy loss), while the second agent tends to explore more (higher variance both in rewards and policy loss). After some other attempts, we can see this is a recurring pattern, even though sometimes the first agent is the one that is exploring more.

Submitted to 38th Conference on Neural Information Processing Systems (NeurIPS 2024). Do not distribute.

Since this is a cooperative game, the optimal policy for both agents is tied to the policy of the other agent, so let us try to implement a mechanism that allows REINFORCE to learn from a "frozen" partner and then we "freeze" the current player and improve on the partner (some kind of switch). In order to maintain consistently the same number of training steps, we also increase the number of epochs

epoch	total reward player 1	policy loss player 1	total reward player 2	policy loss player 2
1	1.0	0.21822035	0.0	
2	-7.0	-4.4614487	0.0	
3	-18.0	-13.935806	-5.0	
4	0.0	0.5641238	0.0	
5	0.0		0.0	-0.27428222
6	-19.0		-3.0	2.5721688
7	0.0		-4.0	0.3031806
8	-15.0		0.0	-0.3076604
9	0.0	0.24935049	0.0	
10	0.0	0.25398988	2.0	
11	-5.0	-4.3706007	-4.0	
12	-4.0	0.41461992	-15.0	
13	1.0		0.0	1.2072344
14	0.0		1.0	-0.17331769
15	-5.0		0.0	-0.37020326
16	-10.0		0.0	-0.38054597

Table 2: First 8 Epochs after first switch implementation

Now we still see a lot of variance in many values, even during the training of the same agent, not only when the active agent "freezes" and the "freezed" agent activates. Let's try to solve the freeze-activate switch with a "smoother" transition: we put an intermediate phase where we train both agents before freezing one of them and we also shorten the "freezing window".

epoch	total reward player 1	policy loss player 1	total reward player 2	policy loss player 2
1	2.0	1.8385005	-3.0	-0.09365102
2	-5.0		-4.0	-3.7806792
3	-15.0	2.021677	5.0	
4	-10.0	-9.47376	-10.0	-6.3929243
5	-10.0		0.0	-0.13836528
6	-10.0	-2.424742	0.0	
7	0.0	0.07146771	0.0	-0.2617634
8	0.0		-5.0	-0.3076604
9	0.0	0.03329641	-5.0	
10	0.0	0.014360208	-25.0	-8.863334
11	-5.0		0.0	-0.22238608
12	0.0	-0.047658503	-5.0	
13	-10.0	-3.4205854	-15.0	-0.025251329
14	0.0		-5.0	-0.004458418
15	0.0	-0.28365055	0.0	
16	0.0	-0.32615718	0.0	-0.014420137

Table 3: First 8 Epochs after the second switch implementation

The results are not very encouraging, we did not obtained the improvement we were hoping for, so let's try changing strategy. There are a lot of zeros on the rewards, this combined with how the environment rewards the agents (positive reward only after a certain action, not after each action, similar problem to the one encountered in Montezuma's Revenge paper).

Until now we separately trained the two agents in order to reduce noise and to obtain a more "stable" environment for REINFORCE. But this poses a problem: since the positive rewards of both agents

depend on collaborating, they both land on a strategy of no cooperation and no defection, that is: stay away from the stag (since if caught they would receive a negative reward) and occasionally land on some plants to receive a small positive reward. This is proven by the fact that in almost every episode 90% to 95% of rewards are 0.

We can try to improve this by including entropy to the policy loss to incentivize the agent to explore a bit more. This should offer more possibilities to earn the positive reward. We define entropy as:

$$E_{\pi(a|s)} = -p(a|s)ln(a|s)$$

We can then compute the final policy loss as:

$$J_T = J(\theta) - \gamma E_{\pi(a|s)}$$

with γ as the entropy coefficient, an hyperparameter that will regulate how much the agent will be incentivize to explore (usually we can keep it between 0.01 and 0.05)

epoch	total reward player 1	policy loss player 1	total reward player 2	policy loss player 2
1	-13.0	-9.022358	-9.0	-12.999744
2	-25.0	-16.273602	-9.0	-7.5871944
3	2.0	-0.6987076	4.0	-3.0613132
4	1.0	-1.3981252	-4.0	-5.616089
5	-4.0	-6.9430213	-14.0	-16.076591
6	-5.0	-5.5814066	-23.0	-21.427134
7	-5.0	-2.3216147	-18.0	-17.621262
8	0.0	-1.9702475	0.0	-1.8299603
9	-21.0	-28.687113	-13.0	-12.80364
10	-5.0	-5.7848153	-20.0	-19.637047

Table 4: First 10 Epochs after adding Entropy to the policy loss

We have reduced the frequency of the "neutral" state, but now rewards are very negative because the agents are not yet able to cooperate. In practice now the agents wander around and usually meet the stag alone, and receive a negative reward. Furthermore, we will try to tackle the problem that without coordination one agent may receive a negative reward even though the action that caused the negative reward was the one of the partner. Previously we relied only on positive rewards to incentivize the agents to cooperate, now we "unify" the baseline networks of both agents in order to obtain an "observer" that tries to detect and incentivize cooperation, called common critic. The common critic conditions the value function $V(s)$over $Q(s, a_1, a_2)$ with s the joint state observation from the 2 agents (in our case both agents see the full environment) and a_1 and a_2 the actions of the 2 agents, thus being capable to discriminate cooperation from defection. Unfortunately due to an issue with tensor shapes I was not able to implement the full common critic, I was able to unify the baseline functions and obtained the following results:

epoch	total reward player 1	policy loss player 1	total reward player 2	policy loss player 2
1	3.0	-1.4595263	4.0	0.21298027
2	-4.0	-6.0123806	-10.0	-8.050568
3	-6.0	-10.141323	-8.0	-14.057734
4	-8.0	-8.78888	-2.0	-0.8003638
5	-15.0	-12.652037	0.0	-2.388352
6	-5.0	2.176623	2.0	-2.0688715
7	-20.0	-2.8272817	1.0	-3.434661
8	10.0	7.2739058	5.0	3.1446419
9	-10.0	-9.759584	-10.0	-7.7842093
10	-25.0	-16.588797	-20.0	-20.011873

Table 5: First 10 Epochs after an initial implementation of the actor critic

3 Conclusions and Future developments

Even though the environment did not presented a large observation space (5 entities distributed over 25 cells), the cooperative aspect of the game proved intractable by REINFORCE, even with multiple upgrades over a direct implementation of the algorithm. Perhaps better results could be obtained by fully implementing the common critic component, and finally considering cooperation not only in the reward level but also directly in the loss functions. And if even the common critic component will fail, we could abandon entirely Reinforce for PPO, Proximal Policy Optimization. in particular we could begin with Independent PPO (IPPO), where we implement both agents to use PPO independently, or Multi Agent PPO (MAPPO), where once again we use a centralized critic to distinguish more clearly cooperation from defection.